

ALPHALIST DEVELOPER BOOTCAMP / HAMBURG / JUNE 16-17, 2026

Intro to the pipeline

Research, plan, implement. The loop your factory runs on.

Malte Burkert

THE PROBLEM

“Build me X.” Then you spend an hour explaining what you **actually meant.**

That is vibe coding. It feels fast and it is the slow path.

The shift

Prompt engineering

Craft the perfect sentence. Hope the result matches.

Context engineering

Give the agent what it needs to check its own work.
The agent is smart enough. The question is whether it has the right context.

THE PATTERN

Research, plan, implement

R

**Research the
codebase**

>

P

Write the plan

>

I

Implement on a branch

Same loop at every level: feature, story, task.

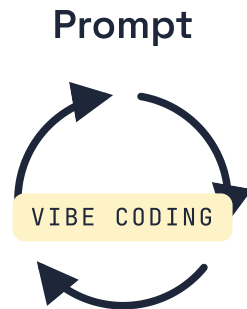
PLANNING

The longer you invest in the plan, the longer the agent runs **unsupervised.**

A plan is a durable artifact. Not a message in a chat.

Keep the agent on a leash

Loop of death



"Not what I meant" Code

Wasted time. Wasted tokens.

Stay in the loop at the cheap end, before any code exists.

On a leash

1 Ask for the plan, not the code

GATE

2 Review, then implement and test

GREEN

3 Commit. Ask what is next

SMALL

You approve the plan, not every keystroke.

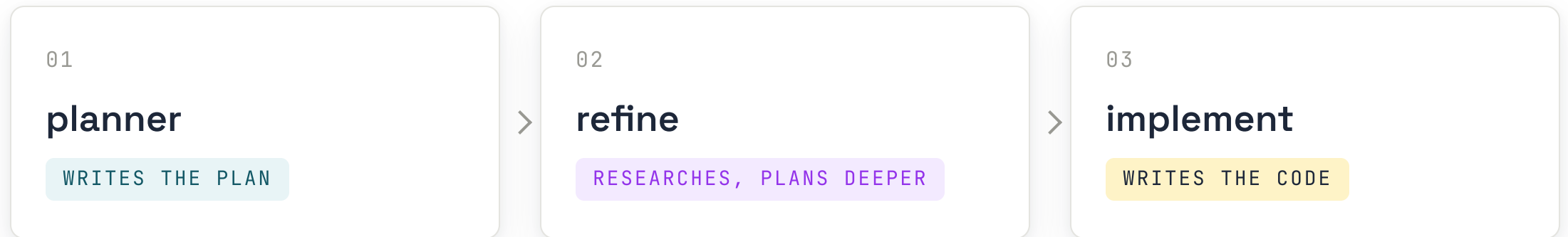
SUB-AGENTS

**A sub-agent does the reading. You keep the
summary.**

Isolated context. Research without the bloat. Not magic, mechanics.

THE BUILDING BLOCK

Connect three skills with a Bean



Each station hands off to the next. The Bean is the contract that connects them.

Rename a heading and the next station breaks

```
# one Bean, three stations append to it
## High-Level Plan      # planner writes this. no file paths.
## Refined Plan         # refine appends this. files, signatures, tests.
## Implementation Log   # implement appends this. branch, commits, SHAs.

# the next station parses by exact-match on the heading.
```

DETERMINISM AND FEEDBACK

What makes it reliable

Exact-match contracts

The next station parses a literal heading. Breaks loud, not silent.

Validate, then run

Missing input? Abort cleanly. No guessing, no fabrication.

Green before commit

Tests gate every step. Two fix attempts, then stop and report.

Hooks and bounds

Guardrails the agent cannot skip. Loops are bounded, escalation explicit.

FOR THE ARCHITECTS WORRIED ABOUT NON-DETERMINISM

Stop fighting the randomness. Engineer **the loop** around it.

Eventual consistency for AI: agree a contract, verify, repeat.

ZOOM OUT

Foundation, pipeline, factory

03 **Factory**
Stations orchestrated end to end

DAY 2

02 **Pipeline**
RPI, sub-agents, contracts, feedback

NOW

01 **Foundation**
Skills, hooks, rules

BLOCK 1

POD BLOCK 2 / 90 MINUTES

Wire your factory's pipeline

Less typing. More delegating and reviewing.



What you'll wire, on the sandbox

```
# a small TypeScript calculator on Node. Beans CLI. Claude Code.  
  
/planner    <idea>      -> Bean with ## High-Level Plan  
/refine     <bean-id>   -> one read-only subagent, ## Refined Plan  
/implement  <bean-id>   -> branch, commits, tests green
```

POD BLOCK 2

90 MIN

EVERYONE WIRES THE SAME PIPELINE

One goal, three starting points

01 **Level 1: run the supplied planner and refine, inspect the Bean.**

Start here if you have no skills of your own yet.

02 **Level 2: drop in the planner you built this morning.**

Reuse your own work.

03 **Level 3: harden refine with one read-only subagent and stricter checks.**

For pairs already ahead. Fallbacks are supplied.

Same goal: a connected pipeline. The implement demo runs at the end.

Your run sheet



CLONE THE SANDBOX

github.com/hackersandwizards/dev-bootcamp-factory-workshop

01 14:30

15 MIN

Factory contract

Inspect a Bean. See the hand-off contract before you build.

02 14:45

50 MIN

Wire your pipeline

Pick your starting point, then build the connected spine.

L1 run supplied

L2 replace planner

L3 harden refine

03 15:35

15 MIN

Implement demo

Trainer runs the supplied /implement on a refined Bean.

04 15:50

10 MIN

Reflect, pick presenter

What a contract saved or broke. Decide who presents tomorrow.

Before you leave the pod

REFLECT

Where did a contract boundary save you, or break you?

COMMIT

Pick who presents at tomorrow's cross-pod show and tell.

Day 2, 12:00 / Cross-pod show and tell

Cover: use case, what worked, what broke

Decide the presenter today, so nobody scrambles tomorrow.